

El Gato | Tic Tac Toe - Enunciado

// 02 MS Server — gato-server

PUERTO 8080

02.1 Tipos de partida

NO_TIME

Sin límite de tiempo

No hay restricción de tiempo por turno. El jugador puede tomarse todo el tiempo que necesite. La partida sólo termina por victoria, empate o abandono.

Campo: `turnTimeoutSeconds: null`

TIMED

Con límite de tiempo

Cada turno tiene un máximo de segundos. Si el jugador en turno no mueve a tiempo, **pierde automáticamente la partida**. El server detecta el timeout en cada consulta de estado o intento de movimiento.

Campo: `turnTimeoutSeconds: 30` (mínimo 10 seg, máximo 300 seg)

Detección de timeout

El server no requiere un scheduler activo. La detección es **lazy**: al recibir cualquier request sobre una partida TIMED en curso, el server calcula si el tiempo del turno actual expiró comparando `lastMoveAt` (o `startedAt` para el primer movimiento) con el momento actual.

- Si el tiempo expiró → la partida pasa a `FINISHED`, el jugador en turno pierde, y se retorna la respuesta con `winner` correspondiente.
- El campo `lastMoveAt` se actualiza en cada movimiento válido.
- El campo `remainingSeconds` puede incluirse en la respuesta de estado para que el cliente informe al jugador.

02.2 Modelo de dominio

ENTIDAD: GAME

- `id` — Long, generado automáticamente
- `playerXId` — Long jugador que crea la partida (rol X)

- `player1Id` — Long, jugador que crea la partida (rol X)
- `player0Id` — Long nullable, jugador que se une (rol O)
- `board` — String de 9 chars: `X`, `O`, `.` (posiciones 0-8)
- `currentTurn` — Enum: `X` | `O`
- `status` — Enum: `WAITING` | `IN_PROGRESS` | `FINISHED`
- `winner` — Enum nullable: `X` | `O` | `DRAW` | `TIMEOUT`
- `gameType` — Enum: `NO_TIME` | `TIMED`
- `turnTimeoutSeconds` — Integer nullable (sólo si `gameType` = `TIMED`)
- `lastMoveAt` — LocalDateTime, actualizado en cada jugada
- `createdAt` — LocalDateTime
- `startedAt` — LocalDateTime, cuando el segundo jugador se une

ENTIDAD: REGISTEREDCLIENT

Representa un MS Cliente autorizado a usar este server. Se crea automáticamente en el primer arranque del cliente.

- `id` — Long, generado automáticamente
- `clientId` — String único, identifica al MS Cliente (ej: `gato-client-1`)
- `clientSecret` — String, hashado con BCrypt en el server
- `baseUrl` — String, URL base del cliente (ej: `http://localhost:8081`)
- `registeredAt` — LocalDateTime
- `active` — Boolean

El server usa esta tabla como `UserDetailsService` para Spring Security: el `clientId` es el username y el `clientSecret` hashado es el password. Migración Flyway obligatoria.

02.3 Reglas de negocio obligatorias

Validaciones de movimiento (en orden)

1. La partida existe → 404 si no.
2. La partida está en estado `IN_PROGRESS` → 422 si está `WAITING` o `FINISHED`.
3. Para partidas `TIMED`: verificar si el turno expiró → si expiró, finalizar y retornar resultado.
4. El jugador es participante de la partida (X u O) → 403 si no.
5. Es el turno del jugador que mueve → 400 con mensaje descriptivo.
6. La posición es válida (0-8) → 400 si está fuera de rango.
7. La casilla no está ocupada → 400 con mensaje descriptivo.

Detección de fin de partida

Tras cada movimiento válido, el server evalúa en orden:

- ¿Línea ganadora? (8 combinaciones: 3 filas, 3 columnas, 2 diagonales) → `winner = X|O`, `status = FINISHED`
- ¿Todas las casillas ocupadas sin ganador? → `winner = DRAW`, `status = FINISHED`
- Ninguna condición → cambiar turno, actualizar `lastMoveAt`, continuar.

02.4 Endpoints

Clic en cada endpoint para ver ejemplos de entrada/salida.

GESTIÓN DE PARTIDAS – /API/GAMES

POST

/api/games



Crea una partida. El creador es el jugador X. Se especifica el tipo NO_TIME o TIMED.

GET

/api/games



Lista partidas. Soporta filtros por status y gameType.

GET

/api/games/{gameId}



Estado completo de la partida. En partidas TIMED incluye remainingSeconds. Detecta timeout lazy.

POST

/api/games/{gameId}/join



Acción: un jugador se une como O. Cambia status a IN_PROGRESS e inicia el reloj si es TIMED.

POST

/api/games/{gameId}/move



Acción principal: registra una jugada. Valida orden completo, actualiza tablero, detecta fin, aplica timeout si aplica.

GET

/api/games/player/{playerId}



Todas las partidas de un jugador en este server (activas e historial) con estadísticas.

REGISTRO DE MS CLIENTES – /API/CLIENTS

POST

/api/clients/register



Endpoint público. Registra un MS Cliente autorizado. Lo llama automáticamente el cliente al arrancar. Si ya existe, actualiza baseUrl y retorna 200.

GET

/api/clients



Lista los MS Clientes registrados en este server. Requiere autenticación de MS. Útil para diagnóstico.

04. Architecture

07. MS Clientes

